BACKEND DEVELOPMENT

Complete Learning Guide

From Zero to Understanding Distributed Chess

Express.js • WebSockets • RabbitMQ • MongoDB • JWT Auth

What You Will Learn

- How the Internet works (HTTP, TCP)
- Building REST APIs with Express.js
- Real-time with WebSockets
- Message queues (RabbitMQ)
- MongoDB database
- JWT authentication
- Microservices architecture

The Chess Project

We will use a real distributed chess backend as our teaching example.

- 3 independent microservices
- Real-time game via WebSockets
- Chess moves validated on engine
- Live leaderboard across sessions
- JWT-secured routes

CHAPTER 1

How the Internet Works

HTTP, TCP/IP, Clients, Servers, and Ports — the foundation of everything

Before writing a single line of backend code, you need to understand what happens when a browser makes a request. Every backend you will ever build exists to serve these requests.

1.1 Client and Server

Every network interaction has two sides:

- Client: the thing that asks. A browser, a mobile app, a curl command, or another server.
- Server: the thing that answers. It listens on a port, accepts incoming connections, and responds.

Real world analogy

Think of a restaurant. You (client) sit down and ask a waiter (server) for food. The waiter takes your request to the kitchen (backend logic, database) and brings back the response (your plate of food). You never walk into the kitchen yourself.

1.2 TCP/IP — The Postal System of the Internet

IP (Internet Protocol) is the addressing system. Every computer has an IP address like 192.168.1.1. TCP (Transmission Control Protocol) is the delivery guarantee — it makes sure packets arrive in order and nothing gets dropped.

When you type chess.com into your browser, here is what happens step by step:

1. DNS lookup: 'chess.com' is translated to an IP address like 104.21.80.1
2. TCP handshake: Your computer and the server do a 3-step handshake (SYN, SYN-ACK, ACK) to establish a reliable connection
3. HTTP request: Your browser sends a formatted text request over that TCP connection
4. HTTP response: The server sends back HTML, JSON, or whatever you requested
5. Connection closed (or kept alive for reuse)

1.3 Ports — Apartment Numbers

An IP address gets you to the right building. A port number gets you to the right apartment. A single server can run many services on different ports:

Port	What runs there by default
80	HTTP (normal websites)

`443`	HTTPS (secure websites)
`3000`	Your Express app (in this project)
`5672`	RabbitMQ message broker
`27017`	MongoDB database
`15672`	RabbitMQ management UI

1.4 HTTP — The Language of the Web

HTTP (HyperText Transfer Protocol) is a text-based request-response protocol. A request looks like this:

```
HTTP Request
GET /api/leaderboard HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

(empty line = end of headers)
```

A response looks like this:

```
HTTP Response
HTTP/1.1 200 OK
Content-Type: application/json

{ "leaderboard": [{ "rank": 1, "username": "chess_master", "wins": 42 }] }
```

HTTP Methods

The method tells the server what action you want:

Method	Meaning and when to use it
`GET`	Fetch data. No body. Read-only. Example: get the leaderboard
`POST`	Create something new. Sends a body. Example: register a user
`PUT`	Replace a resource entirely. Example: replace a user profile
`PATCH`	Partially update. Example: update just the username
`DELETE`	Remove something. Example: delete a game

HTTP Status Codes

The status code is how the server tells you whether things went well:

Code	Meaning

`200 OK`	Success. The request worked.
`201 Created`	Success. A new resource was created (e.g. after register).
`400 Bad Request`	You sent bad data (missing fields, wrong format).
`401 Unauthorized`	You are not logged in or your token is invalid.
`403 Forbidden`	You are logged in but do not have permission.
`404 Not Found`	The resource does not exist.
`409 Conflict`	Duplicate (e.g. username already taken).
`500 Internal Server Error`	The server crashed or has a bug.

CHAPTER 2

Express.js — Building REST APIs

Routing, Middleware, Request & Response — the bread and butter of backend

Express.js is a minimal web framework for Node.js. It listens for HTTP requests and lets you define what code runs for each route. Everything in our chess gateway starts here.

2.1 The Absolute Basics

TypeScript / JavaScript
```
import express from 'express';

const app = express();       // Create the app

// Define a route
app.get('/', (req, res) => {
  res.send('Hello World!'); // Send a response
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Three things to understand from this tiny example:

- `app.get()` — register a handler for GET requests to '/'
- `(req, res)` — req is the incoming request, res is your response object
- `res.send()` — send a response back to the client

2.2 Route Parameters and Query Strings

Routes can capture parts of the URL:

TypeScript
```
// Route parameter — part of the path
app.get('/api/game/:gameId', (req, res) => {
  const { gameId } = req.params;  // e.g. 'A3X9K2'
  res.json({ gameId });
});

// Query string — ?key=value after the path
app.get('/api/users', (req, res) => {
  const { page, limit } = req.query;  // e.g. ?page=2&limit=10
  res.json({ page, limit });
```

```
});
```

In our chess project, game routes use parameters:

game.ts — route parameter in use

```
// From game.ts
router.post('/join/:gameId', authMiddleware, async (req, res) => {
  const { gameId } = req.params;  // Extracts 'A3X9K2' from /join/A3X9K2
  const game = await Game.findOne({ gameId });
  // ...
```

2.3 Middleware — The Assembly Line

Middleware is the most important Express concept. Every request passes through a chain of functions before reaching your route handler. Each function can:

- Read and modify the request (req)
- Read and modify the response (res)
- Call `next()` to pass to the next function in the chain
- End the chain by calling `res.send()` or `res.json()`

Middleware chain

```
// Middleware signature: (req, res, next) => void

// Built-in middleware — parses JSON bodies
app.use(express.json());

// CORS middleware — allows cross-origin requests
app.use(cors());

// Custom logging middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.path}`);
  next(); // Pass to next middleware/route
});

// Route with inline middleware
app.get('/protected', authMiddleware, (req, res) => {
  res.json({ user: req.user });
});
```

The Middleware Flow

Request comes in --> express.json() parses body --> cors() adds headers --> authMiddleware checks token --> Your route handler runs --> Response sent to client. If any middleware calls res.send() without calling next(), the chain STOPS there.

2.4 Our Auth Middleware (from the chess project)

Here is the exact auth middleware from our codebase, explained line by line:

middleware/auth.ts — annotated

```
// middleware/auth.ts
export const authMiddleware = (req: AuthRequest, res: Response, next:
NextFunction) => {
  // 1. Get the Authorization header
  const authHeader = req.headers.authorization;

  // 2. Check it exists and starts with 'Bearer '
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    res.status(401).json({ error: 'No token provided' });
    return; // Stop the chain here — no next()
  }

  // 3. Extract just the token part
  const token = authHeader.split(' ')[1];
  // 'Bearer eyJhbGc...' --> 'eyJhbGc...'

  try {
    // 4. Verify and decode the JWT
    const decoded = jwt.verify(token, JWT_SECRET) as {...};

    // 5. Attach user info to request for later use
    req.user = decoded;

    // 6. Continue to the next handler
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid or expired token' });
    return;
  }
};
```

2.5 Routers — Organising Routes into Files

As your app grows, putting all routes in one file becomes messy. Express Routers let you group related routes in separate files:

Routers

```
// routes/auth.ts
import { Router } from 'express';
const router = Router();

router.post('/register', async (req, res) => { ... });
router.post('/login', async (req, res) => { ... });
router.get('/me', authMiddleware, async (req, res) => { ... });

export default router;
```

```
// server.ts — mount the router
import authRoutes from './routes/auth';
app.use('/api/auth', authRoutes);
// Now: POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
```

2.6 Async/Await and Error Handling

Most backend operations (database queries, file reads) are asynchronous. Always wrap them in try/catch:

```
routes/auth.ts — login route
router.post('/login', async (req: Request, res: Response) => {
  try {
    const { email, password } = req.body;

    // These are async — they take time and could fail
    const user = await User.findOne({ email });
    if (!user) {
      res.status(401).json({ error: 'Invalid credentials' });
      return;
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      res.status(401).json({ error: 'Invalid credentials' });
      return;
    }

    const token = generateToken(user._id.toString(), user.username);
    res.json({ token, user: { id: user._id, username: user.username } });

  } catch (err) {
    // Always catch unexpected errors
    console.error('Login error:', err);
    res.status(500).json({ error: 'Internal server error' });
  }
});
```

CHAPTER 3

MongoDB — Storing Your Data

Documents, Collections, Schemas, and Mongoose ORM

MongoDB is a document database. Instead of tables and rows (like SQL), it stores JSON-like documents in collections. It is flexible, easy to get started with, and pairs naturally with JavaScript/TypeScript.

3.1 Core Concepts

MongoDB term	What it means / SQL equivalent
`Database`	The top-level container. Our project uses 'chess'.
`Collection`	Like a SQL table. We have 'users' and 'games' collections.
`Document`	Like a SQL row. One user, one game. Stored as BSON (binary JSON).
`_id`	Every document gets a unique ObjectId automatically. Like a primary key.
`Schema`	Not enforced by MongoDB itself — we use Mongoose to add structure.

3.2 Mongoose — TypeScript-Friendly ODM

Mongoose is an ODM (Object Document Mapper). It lets you define a schema (the shape of your data) and provides methods to query the database. Here is the User model from our project:

User model

```
// shared/models/User.ts (inferred from auth routes usage)
import mongoose, { Schema, Document } from 'mongoose';

export interface IUser extends Document {
  username: string;
  email:    string;
  password: string;  // Always store HASHED, never plain text
  wins:        number;
  losses:      number;
  draws:       number;
  gamesPlayed: number;
}

const UserSchema = new Schema<IUser>({
  username:    { type: String, required: true, unique: true },
  email:       { type: String, required: true, unique: true },
```

```
  password:    { type: String, required: true },
  wins:        { type: Number, default: 0 },
  losses:      { type: Number, default: 0 },
  draws:       { type: Number, default: 0 },
  gamesPlayed: { type: Number, default: 0 },
}, { timestamps: true }); // Adds createdAt and updatedAt automatically

export const User = mongoose.model<IUser>('User', UserSchema);
```

3.3 CRUD Operations

CRUD = Create, Read, Update, Delete. These are the four database operations you will use everywhere:

```
CRUD with Mongoose
// CREATE — insert a new document
const user = await User.create({
  username: 'chess_master',
  email: 'player@example.com',
  password: hashedPassword
});

// READ — find one document
const user = await User.findOne({ email: 'player@example.com' });
const user = await User.findById('64abc123...');

// READ — find multiple documents
const topPlayers = await User.find({ gamesPlayed: { $gt: 0 } })
  .sort({ wins: -1 })   // Sort descending by wins
  .limit(50)            // Return max 50
  .select('-password'); // Exclude password field

// UPDATE — find and modify
await User.findOneAndUpdate(
  { username: 'chess_master' },           // Filter
  { $inc: { wins: 1, gamesPlayed: 1 } }   // Increment wins and gamesPlayed
);

// DELETE
await Game.deleteOne({ gameId: 'A3X9K2' });
```

3.4 MongoDB Query Operators

MongoDB uses special operators prefixed with $ for conditions and updates:

Operator	What it does
`{ field: { $gt: 5 } }`	Greater than 5

`{ field: { $gte: 5 } }`	Greater than or equal to 5
`{ field: { $lt: 5 } }`	Less than 5
`{ field: { $in: [1,2,3] } }`	Value is in array
`{ $or: [{ a:1 }, { b:2 }] }`	Match condition a OR condition b
`{ $inc: { wins: 1 } }`	Increment wins by 1
`{ $push: { moves: moveObj } }`	Append to an array
`{ $set: { status: 'done' } }`	Set a specific field

3.5 The Game Model in Our Project

The Game document stores everything about a chess match:

Game model schema

```
const GameSchema = new Schema({
  gameId:      { type: String, required: true, unique: true }, // e.g.
'A3X9K2'
  whitePlayer: { type: String, required: true },  // username
  blackPlayer: { type: String },                   // null until someone
joins
  moves: [{                                         // Array of all moves
played
    from:      String,     // 'e2'
    to:        String,     // 'e4'
    player:    String,     // username who made the move
    timestamp: Date
  }],
  status:   { type: String, enum: ['waiting', 'active', 'completed'],
default: 'waiting' },
  winner:   String,        // username, 'draw', or null
  finalFen: String,        // Final board position as FEN string
}, { timestamps: true });

// Usage in ws.ts — saving every move:
await Game.findOneAndUpdate(
  { gameId },
  { $push: { moves: { from, to, player: username, timestamp: new Date() } }
}
);
```

3.6 Connecting to MongoDB

The shared db.ts utility handles connection once and reuses it:

shared/db.ts

```
// shared/db.ts
import mongoose from 'mongoose';

const MONGODB_URI = process.env.MONGODB_URI ||
'mongodb://localhost:27017/chess';

let isConnected = false;

export const connectDB = async () => {
  if (isConnected) return;  // Don't connect twice

  try {
    await mongoose.connect(MONGODB_URI);
    isConnected = true;
    console.log('Connected to MongoDB');
  } catch (err) {
    console.error('MongoDB connection error:', err);
    process.exit(1);  // Crash hard if DB is unavailable
  }
};
```

CHAPTER 4

JWT Authentication

How login works, tokens, and protecting routes

Authentication answers: who are you? Authorisation answers: what are you allowed to do? JWT (JSON Web Token) is the most common way to handle both in modern APIs.

4.1 The Problem JWT Solves

HTTP is stateless. The server remembers nothing between requests. So after you log in, how does the server know it is you on the next request?

Old approach: Sessions. The server creates a session, stores it in memory or a database, gives the client a session ID cookie. Every request, the server looks up that ID. This does not scale well.

Modern approach: JWT tokens. The server creates a signed token containing user info and sends it to the client. The client sends it back with every request. The server just verifies the signature — no database lookup needed.

4.2 What a JWT Looks Like

A JWT is three Base64-encoded parts joined by dots:

JWT structure

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9     <-- Header
.eyJ1c2VySWQiOiI2NGFiYzEyMyIsInVzZXJuYW1lIjoiY2hlc3NfbWFzdGVyIn0  <--
Payload
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c  <-- Signature
```

Decoded, the payload looks like:

JWT payload (decoded)

```
{
  "userId": "64abc123...",
  "username": "chess_master",
  "iat": 1716000000,    // Issued at (Unix timestamp)
  "exp": 1716604800     // Expires at (7 days later)
}
```

Important: JWTs are signed, not encrypted

Anyone can decode a JWT and read the payload. Do NOT put passwords or secrets in it. The signature just proves the server created it and it has not been tampered with. The SECRET key must stay private on the server.

4.3 The Login Flow

Here is what happens when a user logs in, step by step:

6. Client sends POST /api/auth/login with { email, password } in the body
7. Server finds the user in MongoDB by email
8. Server uses bcrypt.compare() to check the password against the stored hash
9. If correct, server calls jwt.sign() to create a token
10. Server sends token back to client
11. Client stores token (localStorage, memory, etc.)
12. Client includes token in every future request as Authorization: Bearer <token>

4.4 Password Hashing with bcrypt

Never store passwords in plain text. bcrypt hashes passwords with a random salt so even identical passwords produce different hashes:

bcrypt usage

```
import bcrypt from 'bcryptjs';

// HASHING (at registration time)
const salt = await bcrypt.genSalt(10);  // 10 = cost factor (higher = slower
= more secure)
const hashedPassword = await bcrypt.hash(password, salt);
// Store hashedPassword in database

// VERIFYING (at login time)
const isMatch = await bcrypt.compare(plainTextPassword, hashedPassword);
// Returns true or false — no need to hash again
```

4.5 JWT in Our Chess Project

JWT generation and verification

```
// middleware/auth.ts
const JWT_SECRET = process.env.JWT_SECRET || 'your-secret-change-in-
production';

// Generate token at login/register
export const generateToken = (userId: string, username: string): string => {
  return jwt.sign({ userId, username }, JWT_SECRET, { expiresIn: '7d' });
  // Token expires in 7 days — user needs to log in again after that
};

// Verify token on protected routes
export const verifyToken = (token: string) => {
  try {
```

```
    return jwt.verify(token, JWT_SECRET) as { userId: string; username:
string };
  } catch {
    return null; // Token invalid or expired
  }
};
```

4.6 WebSocket Auth (Special Case)

WebSockets cannot send custom headers like HTTP requests can. So we pass the JWT as a URL query parameter:

```
WebSocket JWT auth
// Client connects like this:
ws://localhost:3000/game-ws?token=eyJhbGciOiJIUzI1NiJ9...

// Server extracts it in ws.ts:
wss.on('connection', (ws, req) => {
  const url = new URL(req.url || '', `http://${req.headers.host}`);
  const token = url.searchParams.get('token');

  const decoded = verifyToken(token);
  if (!decoded) {
    ws.close(1008, 'Invalid token'); // 1008 = Policy Violation
    return;
  }

  const { userId, username } = decoded;
  // Now we know who this WebSocket connection belongs to!
});
```

CHAPTER 5

WebSockets — Real-Time Communication

Going beyond HTTP: persistent connections, bidirectional messaging

HTTP is request-response: the client asks, the server answers, the connection closes. But for a chess game, we need the server to push updates to clients the moment a move is made. That requires WebSockets.

5.1 HTTP vs WebSockets

HTTP	WebSocket
`One request = one response`	One connection = unlimited messages in both directions
`Client must ask for updates (polling)`	Server can push updates instantly
`New TCP connection per request (usually)`	Single long-lived TCP connection
`Stateless by default`	Stateful — server knows who is connected
`Great for: REST APIs, form submissions`	Great for: chat, games, live data feeds

5.2 The WebSocket Handshake

A WebSocket connection starts as an HTTP request with an Upgrade header. The server agrees, and the connection switches protocol:

WebSocket upgrade

```
// HTTP Upgrade Request (client --> server):
GET /game-ws HTTP/1.1
Host: localhost:3000
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==

// Server accepts (101 Switching Protocols):
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade

// From this point: raw WebSocket frames, not HTTP
```

5.3 WebSocket Events

Both client and server emit events. The key ones are:

Event	When it fires
`connection`	A new client has connected (server-side)
`message`	A message has been received
`close`	The connection was closed (normal or abnormal)
`error`	Something went wrong

5.4 Our WebSocket Server (ws.ts) — Full Walkthrough

Let us go through the WebSocket server in our chess project piece by piece:

State: tracking connections

ws.ts — state

```
// Two Maps track everything in memory:

// 1. username --> WebSocket object
//    Lets us send a message to a specific player by name
const clients = new Map<string, WebSocket>();

// 2. gameId --> { players: string[], playerColors: Map<username, 'w'|'b'> }
//    Lets us know which players are in each game room
const gameRooms = new Map<string, { players: string[]; playerColors:
Map<string,'w'|'b'> }>();
```

Handling a new connection

ws.ts — connection handling

```
wss.on('connection', (ws, req) => {
  // 1. Extract and verify JWT from URL
  const url = new URL(req.url, `http://${req.headers.host}`);
  const token = url.searchParams.get('token');
  const decoded = verifyToken(token);

  if (!decoded) {
    ws.close(1008, 'Invalid token');
    return;
  }

  const { userId, username } = decoded;

  // 2. Register this connection
```

```
  clients.set(username, ws);
  ws.send(JSON.stringify({ type: 'AUTH_SUCCESS', username }));

  // 3. Handle incoming messages...
  ws.on('message', async (msg) => { ... });

  // 4. Clean up when player disconnects
  ws.on('close', () => {
    clients.delete(username);
    for (const [gameId, room] of gameRooms.entries()) {
      room.players = room.players.filter(p => p !== username);
      if (room.players.length === 0) gameRooms.delete(gameId);
    }
  });
});
```

Handling JOIN messages

ws.ts — JOIN handling

```
if (data.type === 'JOIN') {
  const gameId = data.gameId;

  // Look up game in MongoDB to verify it exists and user belongs
  const game = await Game.findOne({ gameId });
  if (!game) { ws.send(JSON.stringify({ type: 'ERROR', message: 'Game not
found' })); return; }

  if (game.whitePlayer !== username && game.blackPlayer !== username) {
    ws.send(JSON.stringify({ type: 'ERROR', message: 'You are not a
participant' }));
    return;
  }

  // Create or update the room
  if (!gameRooms.has(gameId)) {
    const playerColors = new Map();
    playerColors.set(game.whitePlayer, 'w');
    if (game.blackPlayer) playerColors.set(game.blackPlayer, 'b');
    gameRooms.set(gameId, { players: [], playerColors });
  }

  const room = gameRooms.get(gameId)!;
  if (!room.players.includes(username)) room.players.push(username);

  // Restore board state by replaying stored moves
  let currentFen = 'start';
  if (game.moves.length > 0) {
    const chess = new Chess();
    for (const move of game.moves) chess.move({ from: move.from, to:
move.to, promotion: 'q' });
    currentFen = chess.fen();
```

```
  }

  ws.send(JSON.stringify({ type: 'JOINED', gameId, color:
room.playerColors.get(username), fen: currentFen }));
}
```

Handling MOVE messages

```
ws.ts — MOVE handling
if (data.type === 'MOVE') {
  const gameId = data.gameId;
  const room = gameRooms.get(gameId);

  if (!room || !room.players.includes(username)) {
    ws.send(JSON.stringify({ type: 'ERROR', message: 'JOIN the game first'
}));
    return;
  }

  // 1. Persist the move to MongoDB
  await Game.findOneAndUpdate(
    { gameId },
    { $push: { moves: { from: data.move.from, to: data.move.to, player:
username, timestamp: new Date() } } }
  );

  // 2. Find player colors to include in the event
  const playerColor = room.playerColors.get(username);
  const whitePlayer = [...room.playerColors.entries()].find(([,c]) => c ===
'w')?.[0];
  const blackPlayer = [...room.playerColors.entries()].find(([,c]) => c ===
'b')?.[0];

  // 3. Publish to RabbitMQ for chess engine to validate
  publishMove({ ...data, playerId: userId, username, playerColor,
whitePlayer, blackPlayer });
  // Response comes back async via subscribeEvents()
}
```

5.5 Broadcasting Events Back

When the game engine validates a move and publishes a GAME_UPDATE, the gateway receives it and fans it out to all players in that game:

```
Broadcasting to game room
// At the bottom of initWS():
subscribeEvents((event) => {
  // event = { type: 'GAME_UPDATE', gameId, fen, gameOver }
  //    or = { type: 'GAME_OVER', gameId, winner, loser, ... }
```

```
  const room = gameRooms.get(event.gameId);
  if (!room) return;

  // Send to every player in this game's room
  room.players.forEach((playerUsername) => {
    const ws = clients.get(playerUsername);
    ws?.send(JSON.stringify(event)); // ?. safely handles disconnected
clients
  });
});
```

CHAPTER 6

RabbitMQ — Message Queues

Decoupling services, work queues, fanout exchanges, and reliable messaging

RabbitMQ is a message broker. It lets services communicate without being directly connected to each other. Think of it as a postal system for your microservices — services publish messages to queues, other services consume them.

6.1 Why a Message Broker?

Why not just have the gateway call the engine directly (like a regular HTTP request)?

- Direct calls create tight coupling — if the engine is down, the gateway crashes too
- You cannot run 3 engine instances and load-balance fairly without extra work
- If the engine is busy, messages queue up automatically rather than failing
- New services (leaderboard) can subscribe to events without changing existing code

The restaurant analogy again

The waiter (gateway) does not follow the chef (engine) around waiting for your food. They write the order on a ticket (message) and put it on the kitchen pass (queue). The chef picks it up when ready. Multiple chefs can take tickets. New staff (leaderboard) can also watch the pass without bothering the waiter.

6.2 Core RabbitMQ Concepts

Concept	What it is
`Producer`	The service that creates and publishes messages (our gateway)
`Consumer`	The service that reads and processes messages (engine, leaderboard)
`Queue`	A named buffer that holds messages until consumed. Durable queues survive restarts.
`Exchange`	A routing layer. Decides which queues receive a published message.
`Binding`	A link between an exchange and a queue, with optional routing rules.
`Acknowledgement (ACK)`	Consumer tells RabbitMQ 'I processed this successfully, remove it'.
`NACK`	Consumer says 'I failed, put it back in the queue'.

6.3 Two Patterns in Our Project

Pattern 1: Work Queue (for chess moves)

The gateway publishes moves to chess.moves.0, chess.moves.1, or chess.moves.2. Only one engine instance consumes from each queue. This guarantees all moves for a single game go to the same engine (sticky routing), keeping the game state consistent.

Work queue publishing

```
// rabbit.ts — publishing a move
export const publishMove = async (msg: any) => {
  // Hash the gameId to pick the same queue every time
  const index = hashGame(msg.gameId, PARTITIONS); // 0, 1, or 2
  channel.sendToQueue(
    `chess.moves.${index}`,
    Buffer.from(JSON.stringify(msg)),
    { persistent: true }  // Survive RabbitMQ restart
  );
};

// shared/hash.ts — deterministic partition selection
export const hashGame = (gameId: string, partitions: number) => {
  let hash = 0;
  for (let i = 0; i < gameId.length; i++) {
    hash = (hash * 31 + gameId.charCodeAt(i)) % partitions;
  }
  return hash; // Always 0, 1, or 2 for the same gameId
};
```

Pattern 2: Fanout Exchange (for game results)

When the engine finishes processing a move, it publishes to the chess.events fanout exchange. A fanout exchange broadcasts the message to ALL bound queues simultaneously. Both the gateway and leaderboard service receive every event:

Fanout exchange

```
// game-engine/consumer.ts — publishing to fanout
ch.publish(
  'chess.events',  // Exchange name
  '',              // Routing key (empty for fanout — all subscribers get
it)
  Buffer.from(JSON.stringify({
    type: 'GAME_UPDATE',
    gameId: event.gameId,
    fen: result.fen,
    gameOver: result.gameOver
  }))
);

// gateway/rabbit.ts — subscribing to fanout
export const subscribeEvents = async (cb: (msg: any) => void) => {
  const q = await channel.assertQueue('', { exclusive: true });
```

```
  // '' = let RabbitMQ generate a unique queue name
  // exclusive = delete queue when this connection closes
  await channel.bindQueue(q.queue, 'chess.events', '');
  channel.consume(q.queue, (msg) => {
    if (msg) cb(JSON.parse(msg.content.toString()));
  }, { noAck: true });
};
```

6.4 Manual Acknowledgements — Reliable Processing

By default (noAck: true), RabbitMQ considers a message delivered the moment it is sent to the consumer. If the consumer crashes while processing, the message is gone. Manual acknowledgements fix this:

Manual acknowledgements

```
// consumer.ts — with manual acknowledgement
ch.consume(queue, async (msg) => {
  if (!msg) return;

  try {
    const event = JSON.parse(msg.content.toString());
    const result = processMove(event);

    if (result.error) {
      ch.ack(msg); // Invalid move — discard it, don't retry
      return;
    }

    // ... publish results ...

    ch.ack(msg); // Success — tell RabbitMQ to remove it
  } catch (err) {
    console.error('Error:', err);
    ch.ack(msg); // In this project we ack even on error to prevent infinite
loops
    // In production: ch.nack(msg, false, true) to requeue
  }
}, { noAck: false }); // false = manual mode
```

6.5 Connection Retry Logic

In Docker or any environment where services start simultaneously, RabbitMQ may not be ready when your service starts. Always retry:

Retry with backoff

```
const connectWithRetry = async (retries = 5, delay = 3000) => {
  for (let i = 0; i < retries; i++) {
    try {
```

```
      return await amqp.connect('amqp://localhost');
    } catch (err) {
      console.error(`RabbitMQ not ready, retrying (${i+1}/${retries})...`);
      if (i < retries - 1) await new Promise(r => setTimeout(r, delay));
    }
  }
  throw new Error('Failed to connect to RabbitMQ');
};
```

CHAPTER 7

Microservices Architecture

Why split services, how they talk, and the full chess system design

A monolith is one big application that does everything. Microservices is an architecture where you split the application into small, independent services that each do one thing well.

7.1 The Three Services

Gateway Service (Port 3000)

- Entry point for all clients (browsers, apps)
- Handles HTTP REST API and WebSocket connections
- Authenticates users, manages game rooms in memory
- Publishes move events to RabbitMQ
- Subscribes to game results and pushes them to WebSocket clients

Game Engine Service (3 instances)

- Pure chess logic — validates moves using chess.js
- Each instance consumes from one queue (partition 0, 1, or 2)
- Maintains in-memory chess board state per game
- Detects checkmate, draws, stalemate
- Publishes GAME_UPDATE and GAME_OVER events

Leaderboard Service

- Listens for GAME_OVER events on the fanout exchange
- Updates user win/loss/draw counts in MongoDB
- Completely decoupled — can be restarted without affecting games in progress

7.2 The Full Data Flow (A Single Move)

Here is what happens from player clicking a piece to receiving the updated board:

13. Player drags piece from e2 to e4 in the browser
14. Frontend sends WebSocket message: { type: 'MOVE', gameId: 'A3X9K2', move: { from: 'e2', to: 'e4' } }
15. Gateway ws.ts receives it, saves move to MongoDB, publishes to RabbitMQ queue chess.moves.1 (based on hash of 'A3X9K2')

16. Engine Instance 1 consumes the message from chess.moves.1
17. Engine loads the Chess object for 'A3X9K2' from its memory, calls chess.move({from:'e2', to:'e4'})
18. Move is valid — engine gets new FEN string representing board position
19. Engine publishes { type: 'GAME_UPDATE', gameId: 'A3X9K2', fen: '...', gameOver: false } to chess.events fanout
20. Gateway subscribeEvents callback fires, looks up game room for 'A3X9K2', sends GAME_UPDATE to both players via WebSocket
21. Both browsers receive the message and render the new board position

7.3 Shared Package Pattern

In a monorepo, shared code (models, utilities, types) lives in a separate package imported by all services. This avoids code duplication:

Shared package

```
// shared/package.json — exports shared utilities
{
  "exports": {
    "./hash": "./hash.ts",
    "./types": "./types.ts",
    "./db": "./db.ts",
    "./models/User": "./models/User.ts",
    "./models/Game": "./models/Game.ts"
  }
}

// Any service can import:
import { connectDB } from 'shared/db';
import { Game } from 'shared/models/Game';
import { hashGame } from 'shared/hash';
```

7.4 Docker Compose — Running All Services

docker-compose.yml

```
version: '3'
services:
  rabbitmq:
    image: rabbitmq:3-management
    ports:
      - '5672:5672'    # AMQP protocol port
      - '15672:15672'  # Management UI at http://localhost:15672

  mongodb:
    image: mongo:7
    ports:
      - '27017:27017'
```

```
    volumes:
      - mongo_data:/data/db  # Data persists across restarts

volumes:
  mongo_data:
```

CHAPTER 8

Code Walkthrough

Every file explained — read this with the source code open

8.1 server.ts — The Startup File

server.ts — annotated

```
import express from 'express';
import http from 'http';
import { connectDB } from 'shared/db';
import { initRabbit } from './rabbit';
import { initWS } from './ws';
import authRoutes from './routes/auth';
import gameRoutes from './routes/game';
import leaderboardRoutes from './routes/leaderboard';
import cors from 'cors';

const app = express();
const server = http.createServer(app);
// Why http.createServer? Because ws (WebSocket) needs the raw HTTP server,
// not the Express app. They share the same port.

app.use(express.json()); // Parse JSON request bodies
app.use(cors());         // Allow cross-origin requests from the browser

app.use('/api/auth', authRoutes);
app.use('/api/game', gameRoutes);
app.use('/api/leaderboard', leaderboardRoutes);

const start = async () => {
  await connectDB();    // Connect to MongoDB first
  await initRabbit();   // Then connect to RabbitMQ
  initWS(server);       // Start WebSocket server
  server.listen(3000);  // Start listening
};
```

8.2 rabbit.ts — RabbitMQ Connection

This file owns the RabbitMQ connection and exposes three functions: initRabbit (setup), publishMove (send a move to engine), subscribeEvents (receive game results).

Key insight: publishMove uses hashGame to always pick the same queue for the same gameId. This is the sticky routing that ensures game state consistency.

8.3 auth.ts (routes) — Register and Login

The register route does five things: validates input, checks for duplicate username/email, hashes the password, creates the User document, returns a JWT. The login route: finds user, compares password with bcrypt, returns JWT.

Notice how both routes return 401 with a generic 'Invalid credentials' message for login — we deliberately do not reveal whether the email exists or just the password is wrong, for security.

8.4 game.ts (routes) — Game Lifecycle

The game routes handle the pre-game setup:

- POST /create — generates a unique 6-char ID, creates a Game with status 'waiting'
- POST /join/:gameId — second player joins, status becomes 'active'
- GET /history — returns user's games (no moves array, for performance)
- GET /:gameId — returns full game details including moves

After joining, the actual game starts over WebSocket — the HTTP routes only handle setup.

8.5 engine.ts — Chess Logic

engine.ts — annotated

```
export const processMove = (event: any) => {
  const state = getGame(event.gameId);
  // getGame lazily creates a new Chess() board if this gameId is new

  if (event.whitePlayer && event.blackPlayer) {
    setPlayers(event.gameId, event.whitePlayer, event.blackPlayer);
  }

  let move;
  try {
    move = state.chess.move(event.move); // Throws if illegal
  } catch {
    return { error: true }; // Invalid move
  }

  if (!move) return { error: true };

  const gameOver = state.chess.isGameOver();
  const isCheckmate = state.chess.isCheckmate();
  const isDraw = state.chess.isDraw();

  let winner = null, loser = null, winnerColor = null;

  if (gameOver && isCheckmate) {
```

```
    // turn() returns whose turn it is NEXT
    // So if it's white's turn and game is over, white was checkmated, black
won
    winnerColor = state.chess.turn() === 'w' ? 'b' : 'w';
    winner = winnerColor === 'w' ? state.whitePlayer : state.blackPlayer;
    loser  = winnerColor === 'w' ? state.blackPlayer : state.whitePlayer;
  }

  return { fen: state.chess.fen(), gameOver, isCheckmate, isDraw, winner,
loser, winnerColor };
};
```

8.6 leaderboard.ts (service) — Score Updates

The leaderboard service listens for GAME_OVER events. For a checkmate it calls updateScore(winner, loser) which increments wins for the winner and losses for the loser using MongoDB's $inc operator. For a draw, it fetches both players from the Game document and calls updateDraw().

8.7 state.ts — In-Memory Game State

state.ts — annotated

```
import { Chess } from 'chess.js';

interface GameState {
  chess: Chess;           // The chess.js board object
  whitePlayer: string | null;
  blackPlayer: string | null;
}

const games = new Map<string, GameState>();

export const getGame = (id: string) => {
  if (!games.has(id)) {
    // Lazy initialization — create on first access
    games.set(id, { chess: new Chess(), whitePlayer: null, blackPlayer: null
});
  }
  return games.get(id)!;
};

export const removeGame = (id: string) => {
  games.delete(id); // Free memory when game is over
};
```

CHAPTER 9

The 8 Bugs & What They Teach

Learning from mistakes in the codebase

The original codebase had 8 bugs. Understanding them teaches you real-world pitfalls every backend developer faces.

Bug	Location	What it teaches you
1 (Critical)	`game-engine/consumer.ts`	GAME_OVER events were never published. Leaderboard got zero data. Always trace the full event flow end-to-end.
2 (Critical)	`game-engine/engine.ts`	Winner returned on EVERY move, not just game over. chess.turn() after checkmate points to the LOSER. Always understand the API you are calling.
3 (Moderate)	`gateway/ws.ts`	WebSocket clients never cleaned up. Memory leak over time. Always handle disconnect events and clean up state.
4 (Moderate)	`gateway/ws.ts`	crypto.randomUUID() used as global. Not available in older Node. Always import what you use explicitly.
5 (Moderate)	`leaderboard/tsconfig.json`	Missing esModuleInterop. Default imports break. Always check TypeScript config when using npm packages.
6 (Minor)	`engine/consumer.ts`	noAck:true means messages lost on crash. Always use manual acknowledgements for critical processing.
7 (Minor)	`All consumers`	No retry on connect. Crashes if RabbitMQ starts 1 second late. Always add retry logic for external services.
8 (Minor)	`game-engine, leaderboard`	Missing @types/amqplib. TypeScript treats imports as any. Always install type definitions for npm packages.

CHAPTER 10

How to Study This Codebase

A structured learning path from beginner to confident

Week 1 — HTTP and Express

Goal: understand request-response and be able to build simple APIs.

22. Read Chapter 1 and Chapter 2 of this guide
23. Set up Node.js and install Express
24. Build a simple API with 3 routes: GET /hello, POST /echo (returns body), GET /user/:id
25. Add a JSON body parser middleware
26. Read auth.ts (routes) and understand register/login
27. Try calling the register and login endpoints using curl or Postman

Week 2 — MongoDB and Authentication

Goal: persist data and implement JWT auth.

28. Read Chapter 3 (MongoDB) and Chapter 4 (JWT)
29. Set up MongoDB locally or use MongoDB Atlas (free tier)
30. Create a User model with Mongoose
31. Implement register (hash password, save user, return token)
32. Implement login (verify password, return token)
33. Add the authMiddleware and protect a route
34. Test the full auth flow end to end

Week 3 — WebSockets

Goal: build a real-time feature using WebSockets.

35. Read Chapter 5 (WebSockets)
36. Install the 'ws' package
37. Build a simple chat room: client connects, sends { type: 'MESSAGE', text: '...' }, server broadcasts to all connected clients
38. Add JWT verification to the WebSocket connection (token in query param)
39. Read ws.ts in the chess project — trace JOIN and MOVE handling

Week 4 — RabbitMQ and Microservices

Goal: understand message queues and service decomposition.

40. Read Chapter 6 (RabbitMQ) and Chapter 7 (Microservices)
41. Run RabbitMQ with Docker: docker run -p 5672:5672 -p 15672:15672 rabbitmq:3-management
42. Open the management UI at http://localhost:15672 (guest/guest)
43. Write a producer that publishes to a queue
44. Write a consumer that reads from it
45. Try the fanout exchange pattern
46. Read consumer.ts in game-engine — trace the full move processing flow

Questions to Test Yourself

Test your understanding

1. Why is each game's moves always processed by the same engine instance?
2. What happens to a game if the engine instance crashes and restarts?
3. Why does the leaderboard use a fanout exchange instead of a work queue?
4. What is the difference between authMiddleware and verifyToken?
5. Why do we replay all moves from MongoDB when a player reconnects?
6. What would happen if noAck was left as true in the engine consumer?
7. Why does game.turn() return the LOSER in a checkmate situation?
8. What is the purpose of the exclusive queue in subscribeEvents?

Key Files to Read First

File	What to focus on
`shared/hash.ts`	The 5-line hashing function. Understand why multiplying by 31 gives even distribution.
`shared/types.ts`	The message shapes that flow through the entire system.
`gateway/src/ws.ts`	The JOIN and MOVE handlers. Trace what happens step by step.
`game-engine/src/engine.ts`	chess.turn() after checkmate. Why it returns the loser, not winner.
`game-engine/src/consumer.ts`	The full move pipeline: receive -> process -> publish -> ack.
`leaderboard/src/consumer.ts`	How GAME_OVER events become score updates.

You now have everything you need to read, understand, and extend this codebase.

The best way to learn is to run the code, break it, and fix it. Start with Week 1 and build up.